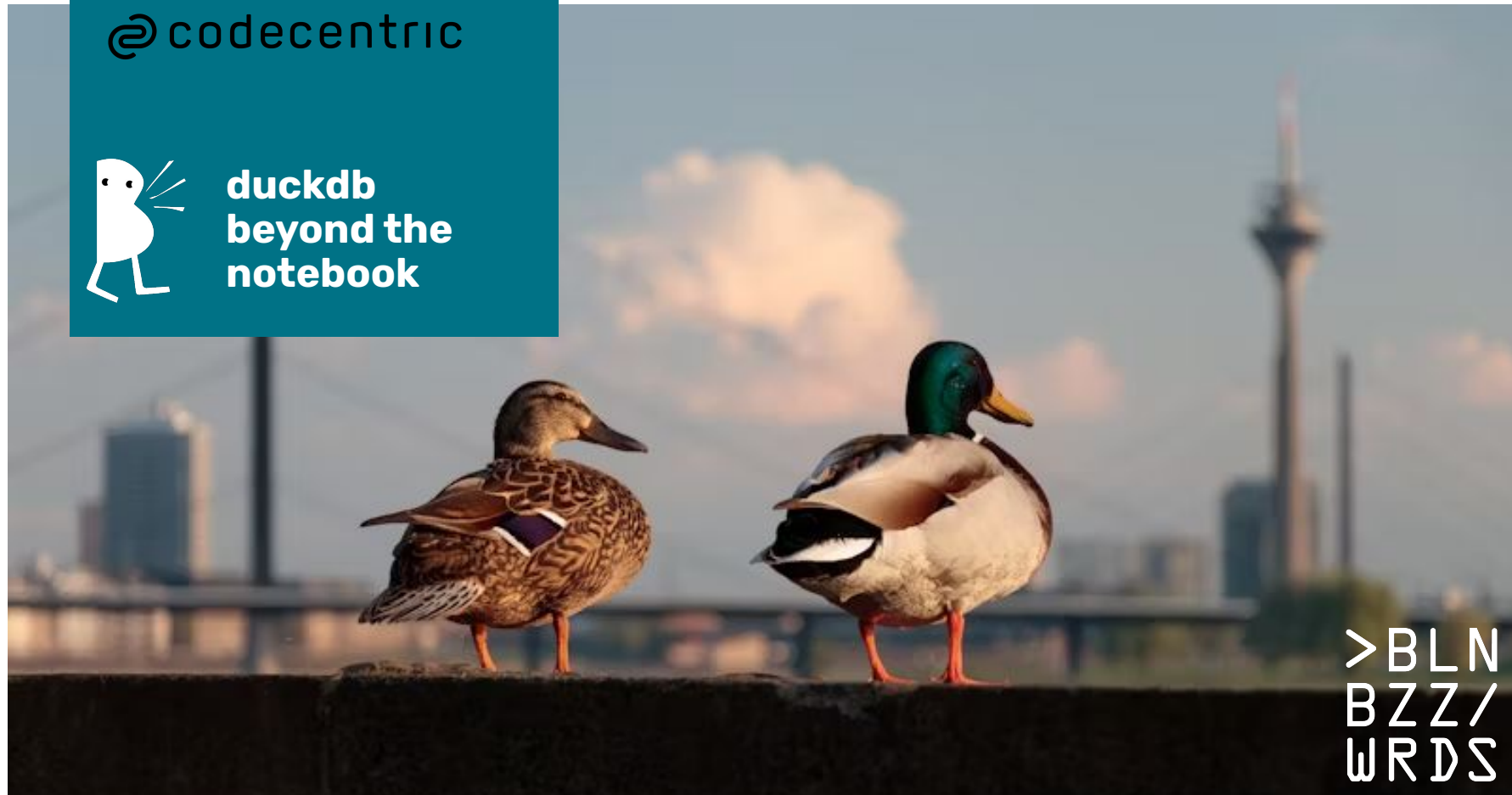


@codecentric



**duckdb
beyond the
notebook**



>BLN
BZZ/
WRDS

DuckDB is a
fast
in-process
analytical
open source
portable
database system

from <https://duckdb.org/>

We start with a demo

```
duckdb --ui
```

Analyse local

```
SELECT *
```

```
FROM read_json("employees.json");
```

```
SELECT department,
```

```
       avg(salary)
```

```
FROM read_json("employees.json")
```

```
GROUP BY all;
```

Analyse remote

```
SELECT *
```

```
FROM read_parquet("https://.../oil-and-gas.parquet");
```

```
SELECT "reporting year" AS year,  
       sum("gas produced, mcf") AS gas  
FROM read_parquet("https://.../oil-and-gas.parquet" )  
GROUP BY "reporting year";
```

```
SELECT *
```

```
FROM read_json("https://api.spacexdata.com/v3/launches")
```

Analyse blob

```
SET s3_region='eu-central-1';
```

```
SET s3_access_key_id='AKIA2RYWYEMUJVK3C4X4';
```

```
SET s3_secret_access_key='';
```

```
SELECT * EXCLUDE payload
```

```
FROM read_json("s3://test-data-matthias/github/*.json.gz")
```

```
SELECT actor.login, actor.url FROM
```

```
read_json("s3://test-data-matthias/github/*.json.gz")
```

Portable

- no external dependencies
- a lot of clients
- every platform - linux, windows, mac



```
pip install duckdb
```



```
org.duckdb:duckdb_jdbc
```



```
install.packages("duckdb")
```



```
npm install duckdb
```

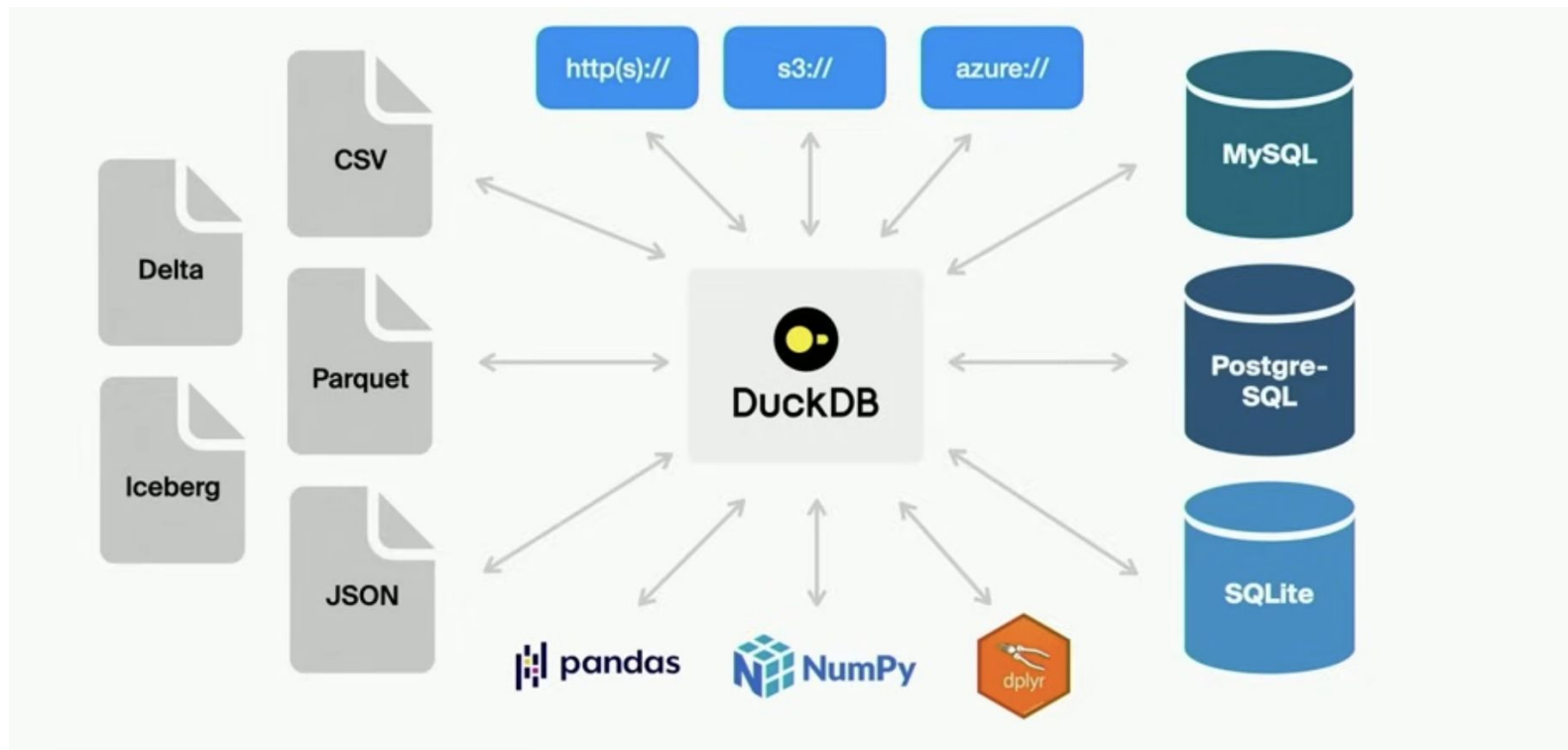


```
brew install duckdb
```



```
cargo add duckdb
```

Flexible



Demo Time

First: Logfiles

duckdb for AI Data Access

demo

<https://github.com/motherduckdb/mcp-server-motherduck>

duckdb ETL in a Lambda

demo

duckdb to replace spark workloads

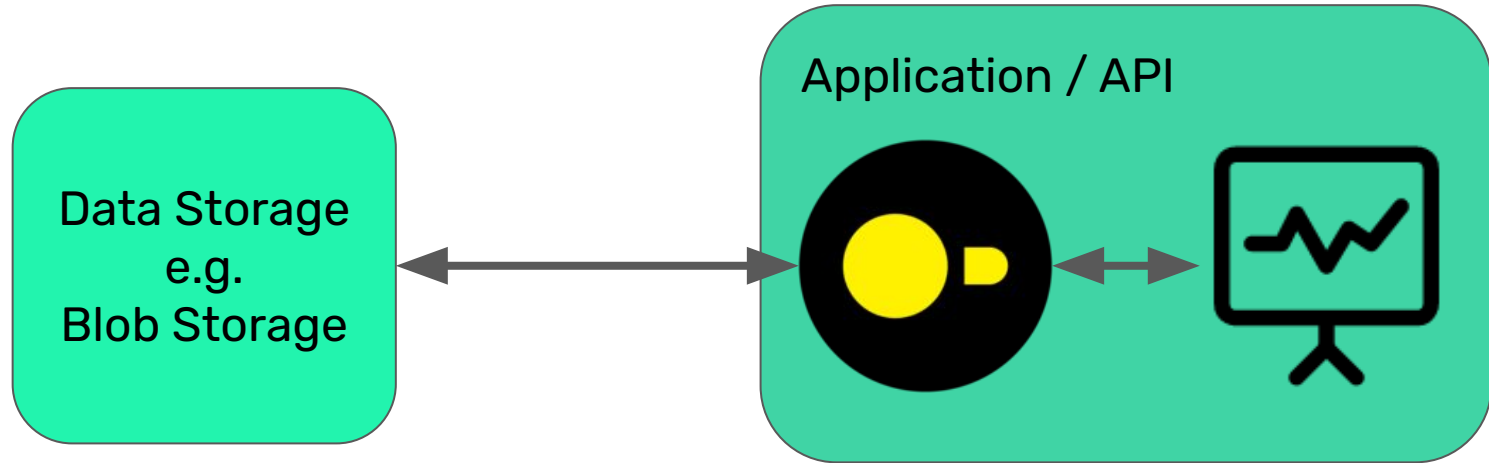
```
from sqlframe import activate

# Activate SQLFrame to run directly on DuckDB
activate(engine="duckdb")

from pyspark.sql import SparkSession
session = SparkSession.builder.getOrCreate()
```

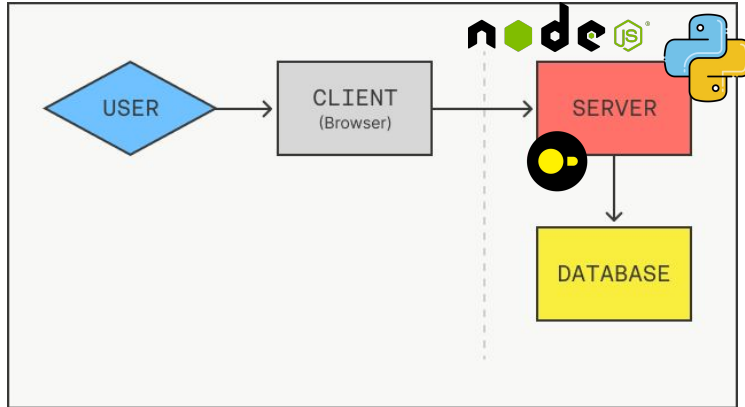
Data Applications / API

The query engine for a fast user experience



The “usual” way:

traditional 3-tier



duckdb in the backend

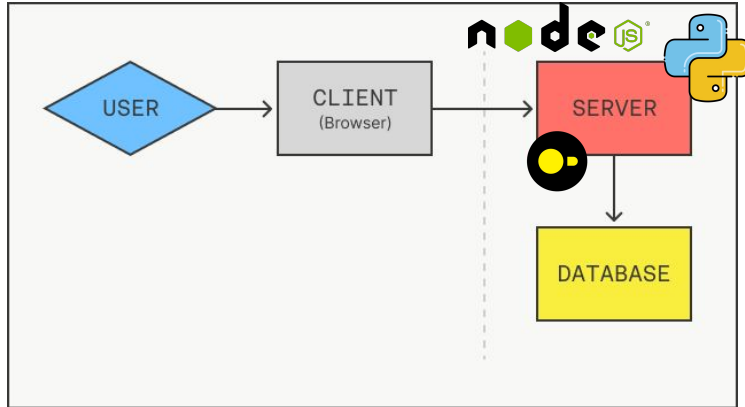
demo

duckdb in the backend

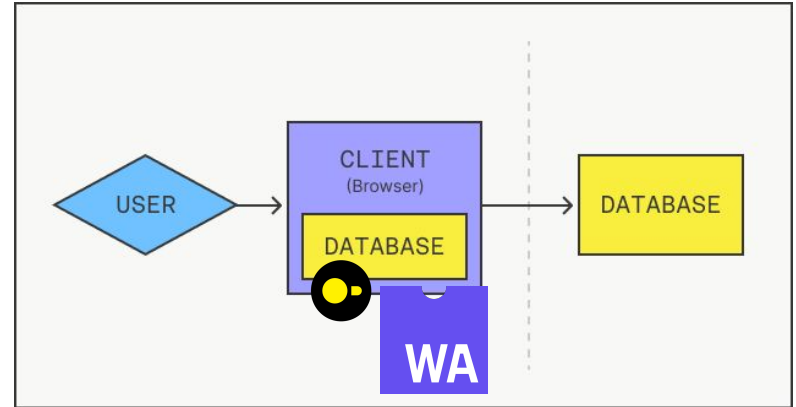
- Query Engine in the backend
- Storage could be (cheap) object storage
- Backend could be run
 - once
 - or even for every client
 - scaled to zero
- Still the classic three tier architecture
 - Storage
 - Backend
 - Frontend
- But query engine has been decoupled from the storage

Going Beyond 3 Tier Data Architectures" ?

traditional 3-tier



client side computing



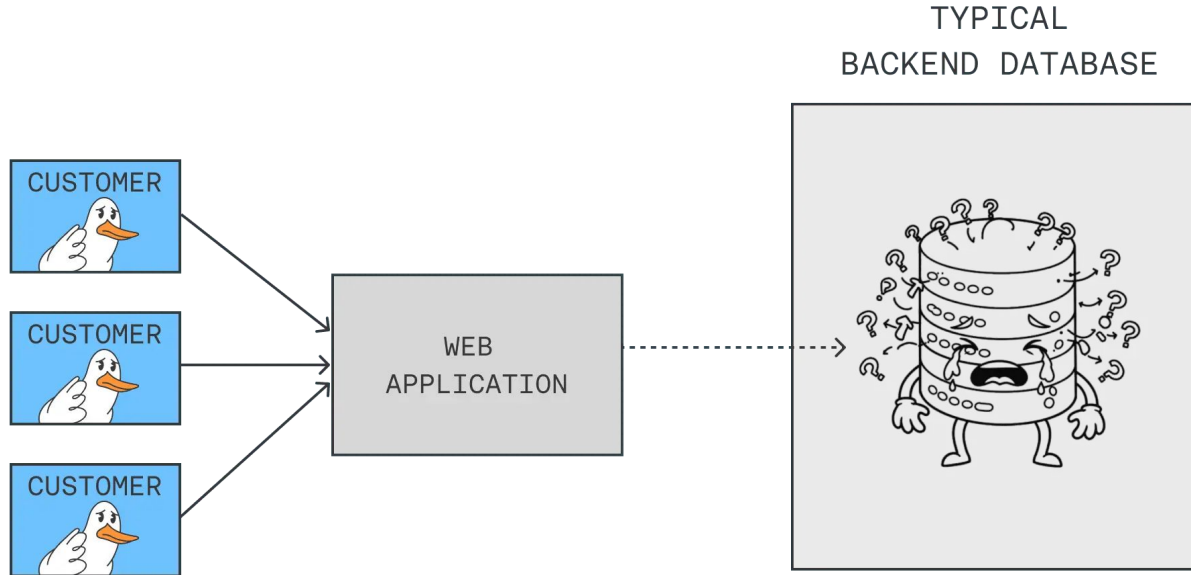
Data App in the browser with WASM

demo

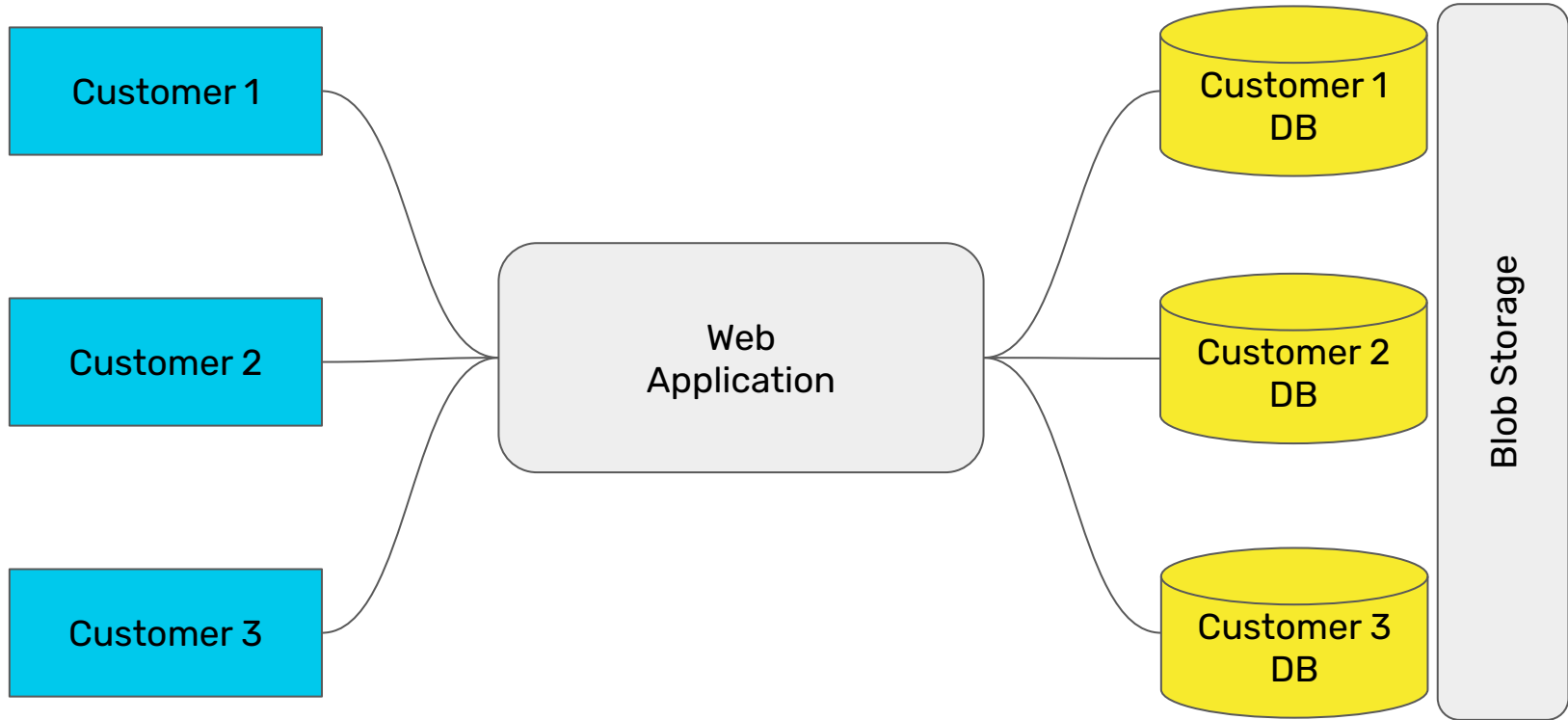
WASM Data Application

- Plenty of examples and applications already
- TypeScript + WebAssembly library
- WebShell
- Caveats:
 - Single Threaded & WASM Memory Limits
 - Beware of CORS
 - No local file access
 - Some extensions not available

Typical Analytical Applications



Scaling a Analytical Applications

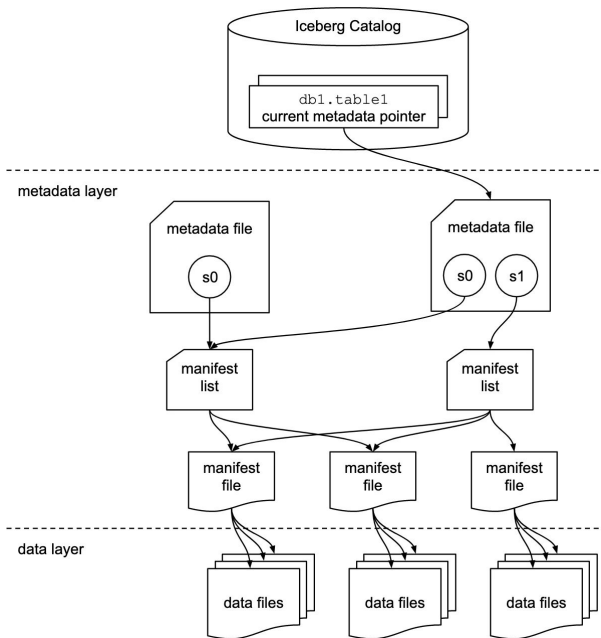
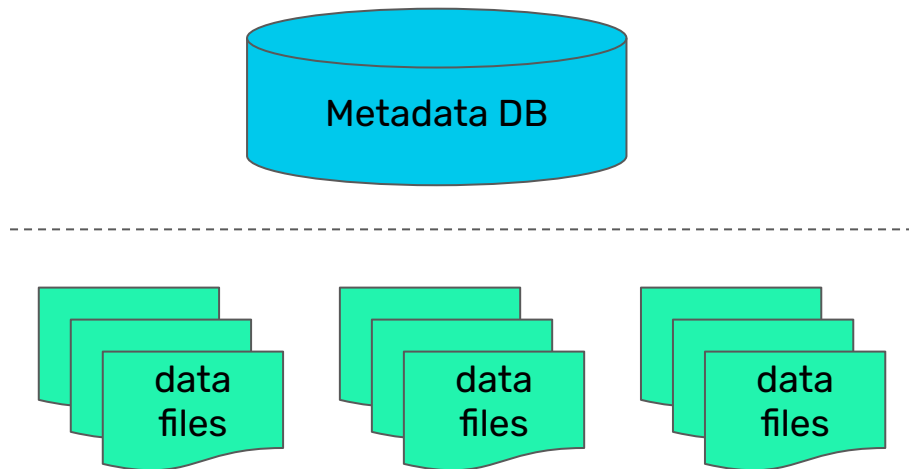


**It's not only a library
(anymore)**

ducklake - “Lakehouse” format



- duckdb's take on a modern table format
- Snapshots, transactions, time travel, schema evolution ...



Quack Protocol (beta)

Clients

DuckDB client + Quack extension

```
SELECT * FROM orders LIMIT 5;
```

id	product	total
1	widget	42.00

DuckDB client + Quack extension

```
INSERT INTO orders VALUES  
  (3, 'tool', 29.99);  
1 row inserted.
```

Server

DuckDB server + Quack extension

```
CALL quack_serve();
```

🔄 listening on port 9494

🔗 2 active connections

📄 catalog: my_database.duckdb

quack:



quack:

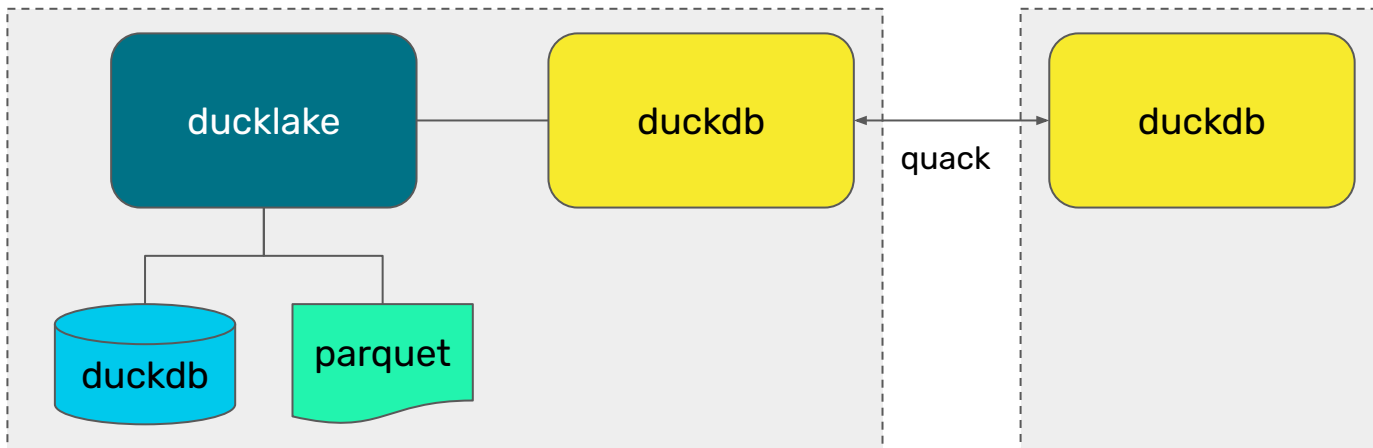


duckdb as a server database for analytics

demo

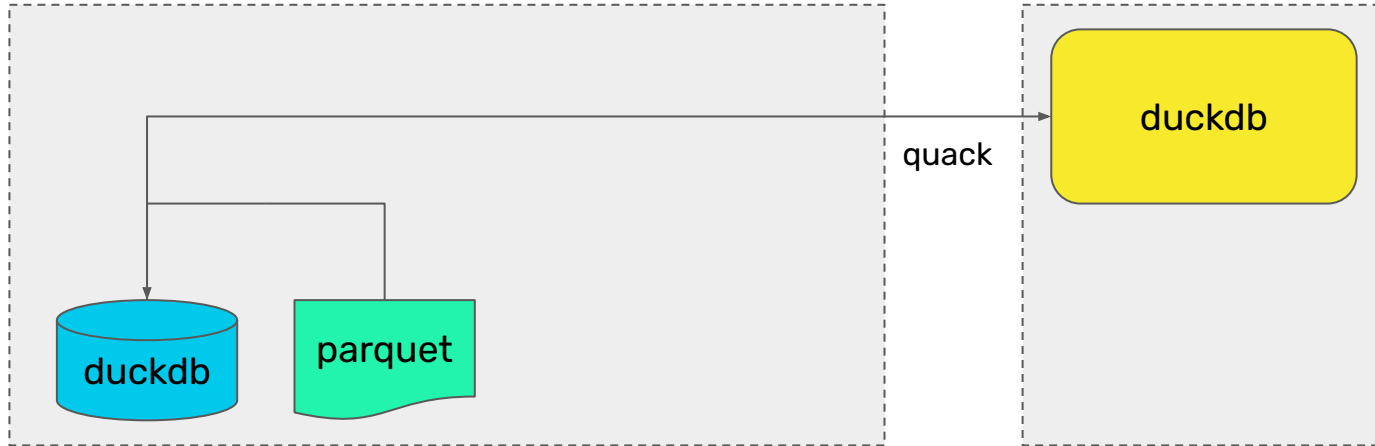
Data Application

- Using duckdb as server database
- Quack + ducklake for multi user access & scaling



Alternative

- connect to duckdb catalog database
- client executes the query
- before: server duckdb executed the query, returns resultset



Why use it?

- made for analytical workloads
- duckdb power on the client
 - post process before inserting
 - aggregate on the client
 - create temp table on the client for caching
 - create a local read replica
(replication is planned for quack)
- compared to “duckdb embedded”
 - supports multiple source of updates
 - easier to operate than “distribute files”

What to do with duckdb?



Local prototyping & exploration

01 - Analyse local

 Run  memory ▾

```
1 SELECT *
2 FROM read_json("/Users/matthiasniehoff/dev/git/duckdb/data/simple/employees.json");
3
```

 Run  memory ▾

```
1 SELECT department,
2       avg(salary)
3 FROM read_json("/Users/matthiasniehoff/dev/git/duckdb/data/simple/employees.json")
4 GROUP BY all;
```

Single Node Data Processing Engines



single node
not bigger than
memory



single node
bigger than
memory

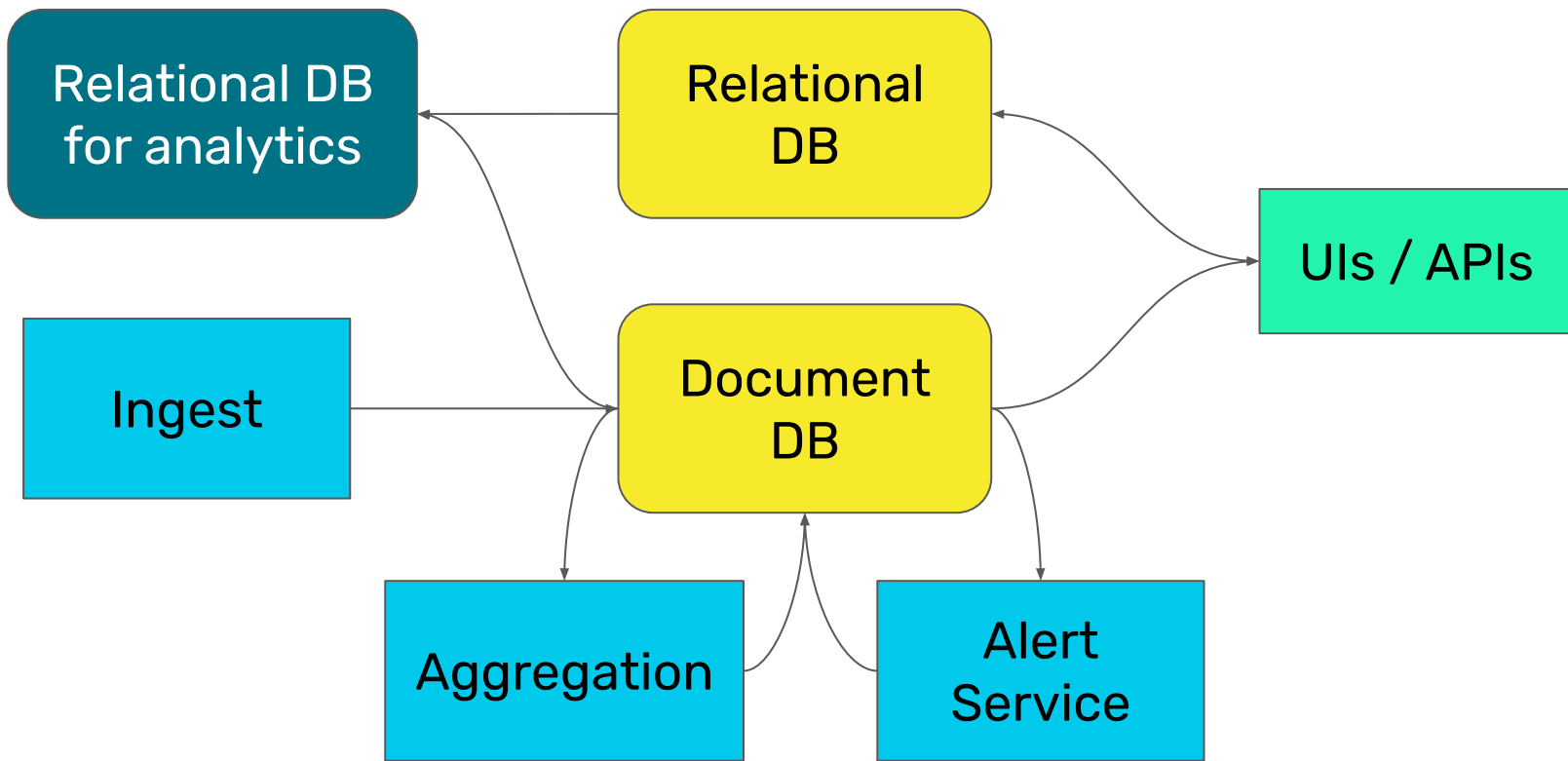


multi node
linear
scaling

Simpler data driven applications

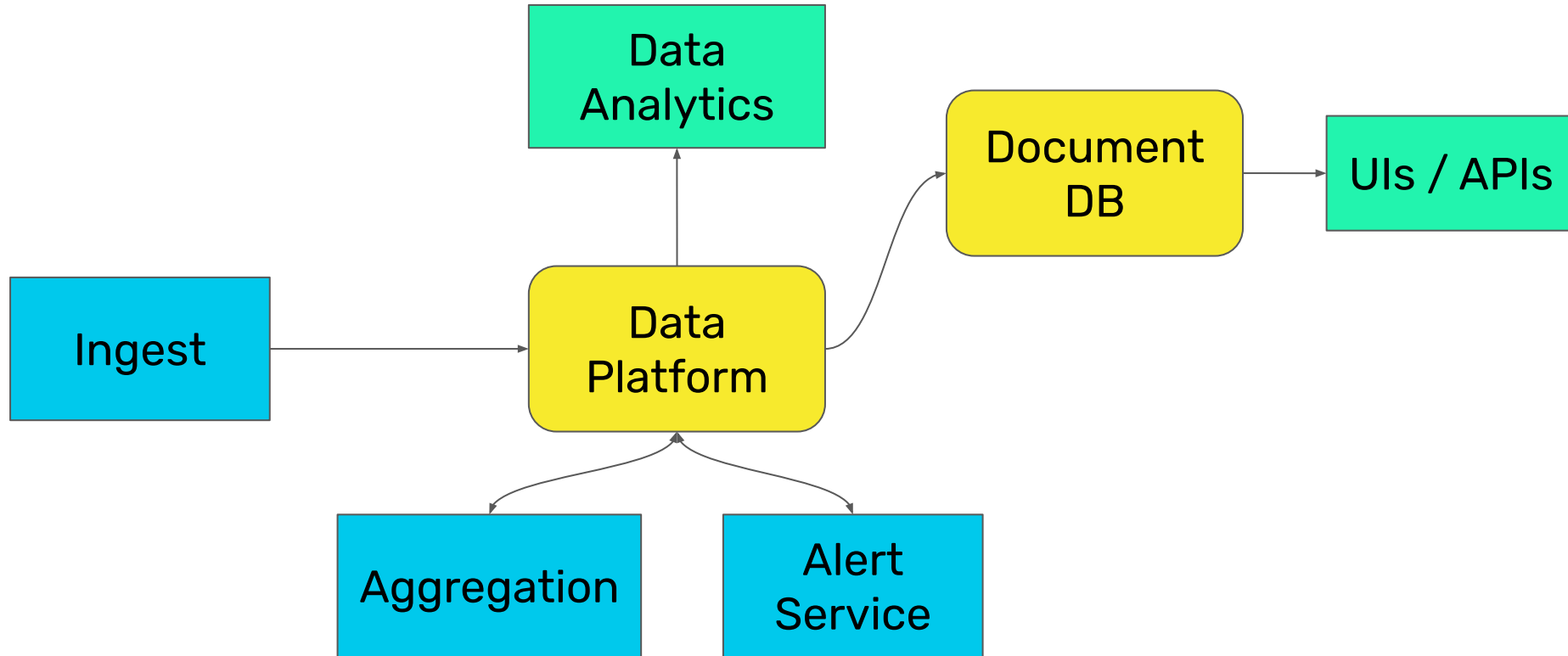


What we had

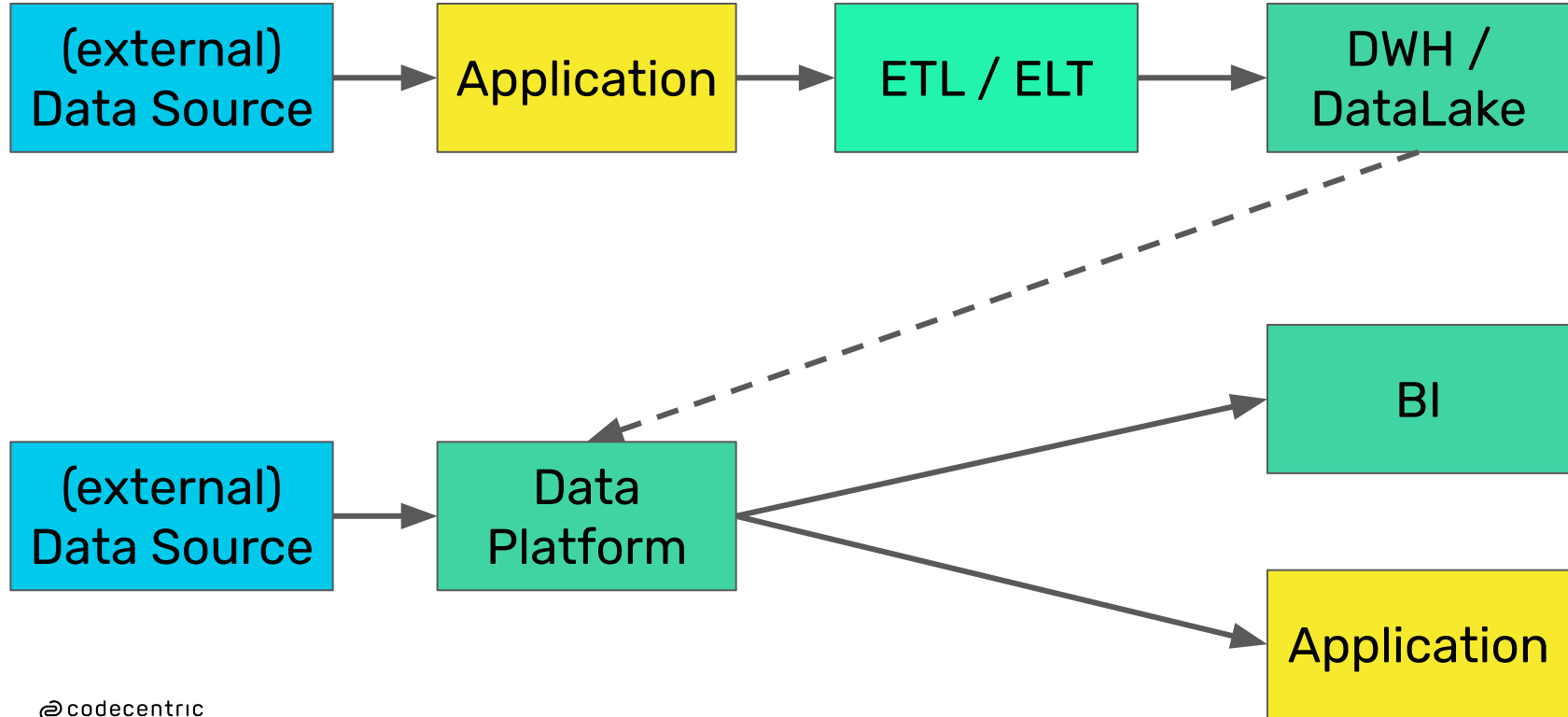


**The web application is actually a
data platform with an UI**

What we have now



Shift left in data flows



A brown paper takeout bag with two handles sits on a concrete surface. A white receipt is attached to the front of the bag. The bag features a large, stylized black logo that resembles a 'D' with wavy lines underneath. In the bottom right corner of the bag, the word 'SHAKE' is visible next to a small circular logo. The background is a blurred concrete wall and floor.

Takeaways

Conclusion

- **duckdb is not only for analysts**
- **used to build data driven / analytical applications**
- **powers data pipelines in large companies**
- **versatile, flexible and friendly to use**
- **might go for “basic” transactional workloads eventually**



Repository & Slides



**mniehoff/
duckdb-beyond-the-notebook**

**We support you with
your data projects**

Feel free to contact us!



codecentric AG
Hochstraße 11
42697 Solingen



Matthias Niehoff
Principal Data Architect
matthias.niehoff@codecentric.de
www.codecentric.de

@codecentric

